

PRÁTICAS DE PROGRAMAÇÃO

Programação Estruturada e Listas em Python

Funções, procedimentos, escopo de variáveis e estruturas de dados

PARTE 1 · CAPÍTULO 1

Programação estruturada

Organizar o código-fonte em funções e procedimentos, dando a cada trecho a sua responsabilidade.

Por que dividir o código em blocos?

Núcleo único

- Código extenso em um só bloco
- Instruções repetidas para tarefas idênticas ou similares
- Difícil de entender e de manter



Blocos com responsabilidade

- Subprogramas invocados na ordem e na quantidade que o desenvolvedor definir
- Facilita refatoração: melhorar o código sem mudar as funcionalidades
- Reutilização e legibilidade

Modularizar = escalabilidade + manutenibilidade: partes desenvolvidas, testadas e aprimoradas de forma independente.

Função: bloco reutilizável que retorna um valor

```
def nome_da_funcao(param1, param2):  
    """Docstring opcional"""  
    # Bloco de código da função  
    return resultado
```

```
def saudacao(nome):  
    return f"Olá, {nome}! Bem-vindo  
    ao mundo da programação."
```

- Como na matemática: toda função tem um retorno — o resultado do processamento.
- Pode receber parâmetros com os elementos usados na execução.
- Organiza o código: mais modular, reutilizável e legível.

Sintaxe em 3 partes: def +
nome(parâmetros) → corpo indentado →
return.

Do bloco único às funções: exemplo do livro

```
def calcular_media(n1, n2, n3):  
    return (n1 + n2 + n3) / 3  
  
def verificar_aprovacao(media,  
    media_aprovacao=7.0):  
    return "Aprovado" if media >= \  
        media_aprovacao else "Reprovado"  
  
def exibir_resultado(nome, media, res):  
    print(f"Aluno: {nome}")  
    print(f"Média: {media:.2f}")  
    print(f"Resultado: {res}")  
  
media = calcular_media(8.0, 7.5, 6.0)  
resultado = verificar_aprovacao(media)  
exibir_resultado("João", media, resultado)
```

1

Três responsabilidades

Calcular, verificar e exibir — cada uma em seu bloco.

2

Parâmetro default

`media_aprovacao=7.0` é usado se nada for informado na chamada.

3

Bloco principal enxuto

Só invoca as funções na ordem do processamento.

Função × procedimento

FUNÇÃO

- Calcula e retorna um valor (return)
- Ideal quando o resultado será usado em outro ponto do programa

PROCEDIMENTO

- Executa ações sem necessariamente retornar algo (ex.: `exibir_resultado`)
- Em Python é uma função sem return explícito

Detalhe de Python: mesmo sem a expressão `return`, o procedimento devolve o valor **None** ao final da execução.

Escopo: onde cada variável existe

```
mensagem_global = "Olá, bem-vindo!"

def saudacao(nome):
    mensagem_local = f"Olá, {nome}!"
    print(mensagem_global) # ok
    print(mensagem_local)  # ok

saudacao("Lucas")
```

```
print(mensagem_local)
# NameError: variável local fora da função!
```

- Global: declarada fora das funções; acessível de qualquer parte do código.
- Local: nasce dentro da função e só existe no bloco dela.
- Recursos do subprograma são liberados ao final da execução.

Variável global: conveniente, porém arriscada

1

Depuração difícil

Pode ser modificada de qualquer parte → achar a origem de um erro vira caçada.

2

Dependências ocultas

Funções ficam menos previsíveis e menos reutilizáveis.

3

Sobrescrita acidental

Comportamentos inesperados e difíceis de corrigir.

Como evitar

- Priorizar variáveis locais
- Passar argumentos explicitamente para as funções
- Encapsular estados em classes

Curiosidade: funções em Python são cidadãos de primeira classe — podem ser atribuídas a variáveis, passadas como argumento e retornadas por outras funções (base de lambda, map, filter e de frameworks como Flask e Django).

PARTE 2 · CAPÍTULO 2

Listas

Estruturas de dados para armazenar, acessar e processar conjuntos de valores com uso inteligente da memória.

Estruturas unidimensionais

Homogênea (mesmo tipo)

10	20	30	40	50
0	1	2	3	4

Heterogênea (tipos diferentes)

'sol'	32	2.4	'lua'	True
0	1	2	3	4

Índice: começa em 0 e termina em $n-1$

```
numeros = [10, 20, 30, 40, 50]
print(numeros[2])
# 30
```

```
dados = ["sol", 32, 2.4,
         "lua", 64, True, -1.2]
print(dados[3])
# lua
```

Cada elemento é localizado pelo índice — dá para achar um dado (ou garantir que ele não existe) rapidamente.

Inserindo elementos

```
numeros = [10, 20, 30]
numeros.append(40)          # adiciona no final
# [10, 20, 30, 40]

numeros = [10, 20, 30]
numeros.insert(1, 15)       # insere 15 na posição 1
# [10, 15, 20, 30]

numeros = [10, 20, 30]
numeros.extend([40, 50, 60]) # vários valores ao final
# [10, 20, 30, 40, 50, 60]
```

append() um item no fim · **insert()** em posição escolhida · **extend()** vários itens no fim

Removendo e alterando elementos

```
numeros = [10,20,30,40,30,50]
numeros.remove(30)
# remove a 1ª ocorrência
# [10, 20, 40, 30, 50]
```

```
numeros = [10,20,30,40,50]
del numeros[1] # pelo índice
# [10, 30, 40, 50]
```

```
numeros.clear() # esvazia
# []
```

```
numeros = [10,20,30,40,50]
numeros[2] = 35 # altera
# [10, 20, 35, 40, 50]
```

remove(valor)

busca do início ao fim e tira a primeira ocorrência.

del lista[índice]

remove pela posição.

Buscando elementos

```
numeros = [10, 20, 30, 40, 50]
if 30 in numeros:
    print("30 está na lista")
```

```
for i in range(len(numeros)):
    if numeros[i] == 30:
        print("Índice:", i)
        break
```

- Operador in: jeito mais usado pela comunidade para saber se um valor existe.
- Busca personalizada: loop percorre elemento a elemento comparando com o alvo.
- len() conta os elementos; break interrompe o loop ao encontrar — evita verificações desnecessárias e melhora a eficiência.

Listas e funções: passagem por referência

```
def adicionar_elemento(lista):  
    lista.append(100)
```

```
numeros = [10, 20, 30]  
adicionar_elemento(numeros)  
# numeros → [10, 20, 30, 100]
```

```
def sem_modificar(lista):  
    nova = lista.copy()  
    nova.append(100)  
    return nova
```

- A função recebe a referência da lista original — não uma cópia. Modificou lá dentro, modificou a original.
- `copy()` desvincula: cria uma nova lista e preserva a original.
- O mesmo cuidado vale no retorno: devolver a referência ou a cópia muda o efeito sobre os dados.

Estruturas bidimensionais: linhas × colunas

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(matriz[1][2])  # 6
```

```
for linha in matriz:  
    for elemento in linha:  
        print(elemento, end=" ")  
    print()
```

1	2	3
4	5	6
7	8	9

`matriz[1][2]` → linha 1, coluna 2

Percorrer 2D exige dois laços em cascata: um para as linhas, outro para as colunas.

O que levamos de hoje

1

Modularize

Funções e procedimentos dão responsabilidade a cada trecho e facilitam manutenção e reutilização.

2

Respeite o escopo

Local vive no bloco; global é acessível em todo lugar — e por isso é arriscada.

3

Domine a lista

Inserir, remover, alterar e buscar: `append`, `insert`, `extend`, `remove`, `del`, `clear`, `in`.

4

Referência ≠ cópia

Lista entra na função por referência; `copy()` protege a original. 2D = listas aninhadas.